

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

10. Soros kommunikációs vonalak II.

Írta: Csapó Dániel

Lektorálta: Preinsperger Máté

A 8. tananyagrészen olvashattatok az I²C és SPI felépítéséről (, működéséről). Folytatva ezt a témakört, további három gyakran használt soros kommunikációs formát mutatok be nektek. Az elsőről, az UART-ról, már olvashattatok a Kristálytisza Elektronika első tananyagának 18. részében, de ha mégsem, ne csüggedjete, mindenről ejtünk szót! Egy gyakorlati példát követve kicsit elméleti módon átrágjuk magunkat a CAN és Univerzális Soros Buszokon (USB) is.

ASZINKRON KOMMUNIKÁCIÓ

Az eddig megismert soros portok közül szinte mind közös órajel segítségével, szinkron módon végezték az információcserét. Nos, ez most megváltozik! A következőkben bemutatott protokollok mindegyike aszinkron, tehát nincsen sem órajelvezeték, sem pedig külső tényező, ami alapján egymáshoz tudnának igazodni az eszközök belső ütemezései. Ebből kifolyólag szükségük lesz valamilyen módszerre, amivel ezt a feladatot egymás között megoldják, ezért nincs szükség órajel átvitelére.

De hogyan is működik az aszinkron kommunikáció? Az adó és vevő felek is a saját órajelüket használják a kommunikáció szinkronizálásához. A kommunikációs csatorna egységes ütemezését különböző időközönként a fogadó fél órajelének igazításával érik el. Természetesen változhat a szerepkör az eszközök között, de törvényszerű, hogy mindig a küldő órajelére szinkronizáljuk rá a fogadó oldalt. Ezzel a módszerrel lehet biztosítani a stabil, nagy sebességű kommunikációt. Kis mértékben gyakran eltér a két oldal időzítése, amit az adott kommunikációs forma vagy tolerál, vagy hibát jelez.

A korábbi tananyagrészen nem került elő, mert kevés relevanciája van az ott bemutatott kommunikációk esetében, azonban fontos, hogy ismerjük a fogalmakat: elkülönítünk a kommunikációs protokollok esetén fizikai és adatkapcsolati réteget. Könnyű ezt bemutatni az I²C és SPI esetén, ahol az összeköttetést biztosító "vezetékek" a fizikai réteg részei, az időzítés és az adat beérkezése pedig az adatkapcsolati rétegé. Nehezebb azonban átlátni ezt a megkülönböztetést az UART és USB esetében, különösen, hogy gyakran a szakirodalom is téveszt, ezt a bemutatást a későbbiekben végzem el.

A bemutatott kommunikációs formákra többször utalok majd protokollként és szabványként, azonban ezek a szavak rendelkeznek többletjelentéssel is. A tananyagrészen szereplő valamennyi soros portra sok másik magasabb szintű, összetettebb protokoll vagy szabvány épít, és egészíti ki azokat. Ilyen például az UART-ra épülő Modbus RTU, vagy a CAN-re épülő CANopen protokoll.

Kitekintés:

Bitbanging

A bitbanging egy szleng kifejezés a szoftverből megvalósított kommunikációra. Ilyen esetekben nincsen dedikált hardvereszköz, amely figyelne az időzítésekre vagy a jelek sorrendjére, így mindezt közvetlenül a CPU végzi regisztermanipulációval. Viszonylag jól ismert technikáról van szó, azonban a mai mikrovezérlők nem szűkölködnek perifériákban, így leginkább kényszerből alkalmazzák, ha az eszközön mégis kevesebb kommunikációs periféria van, mint amit a feladat igényelne. Ilyenkor célravezető lehet az adott periféria szoftveres emulálása (másolja a viselkedését), azonban ez természetesen sokkal több számítási teljesítményt igényel, mintha a célhardvert használnánk. A szoftveres megvalósítás nagy sebességű adatfolyam esetén megbízhatatlanabb, mint az erre kialakított perifériák, mivel az adatok feldolgozását azonnal el kell végezni, nincsen olyan puffer, mint a hardveres interfészek esetén. Példának okáért egy 1 MHz-es SPI periféria SCLK jelének regiszterét 2 MHz-el kell írunk (ki- és bekapcsolás). Csak ezt figyelembe véve a bitbanging hatalmas terhelést ró a processzorra.

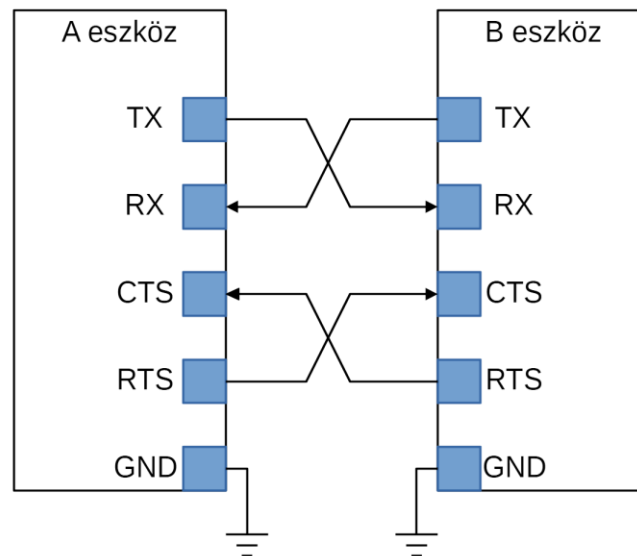
UART

A betűszó feloldása "univerzális aszinkron adó-vevő" (Universal Asynchronous Receiver-Transmitter), egy kommunikációs adatkapcsolati réteget meghatározó protokoll. Maga az UART annak az áramköri perifériának a megnevezése, ami lebonyolítja az aszinkron soros kommunikációt, azonban hasonlóan az I²C-hez és SPI-hoz a megnevezés ma a kommunikáció folyamán küldött adat formáját jelenti leginkább. Általában 2-4 jelet alkalmazunk a kommunikáció kivitelezéséhez, melyek az alábbi táblázatban láthatóak:

Rövidítés	Angol név	Magyarázat	Mivel van összekötve a fogadó oldalon?
TX	Transmit	Adat küldése	RX
RX	Receive	Adat fogadása	TX
CTS	Clear To Send	Adat fogadására való készenlét (opcionális)	RTS
RTS	Ready To Send	Adat küldésére való készenlét (opcionális)	CTS

1. táblázat - UART kommunikációhoz tartozó jelek

Két pár jelet szoktunk egymással összekötni: a TX-RX és CTS-RTS jeleket. Az utóbbiak nem feltétlenül szükségesek a teljes kommunikációhoz, úgynevezett Flow control-t valósítanak meg, mellyel biztosítani lehet, hogy minden keret a megfelelő időben kerül közvetítésre. Ezekkel a jelekkel tudja a két eszköz jelezni, hogy több időre van szüksége a küldendő/fogadott adat feldolgozásához. A CTS és RTS jelek az RS232 (részletesebben a következő kitekintésben) szabvány szerves részét képezik.



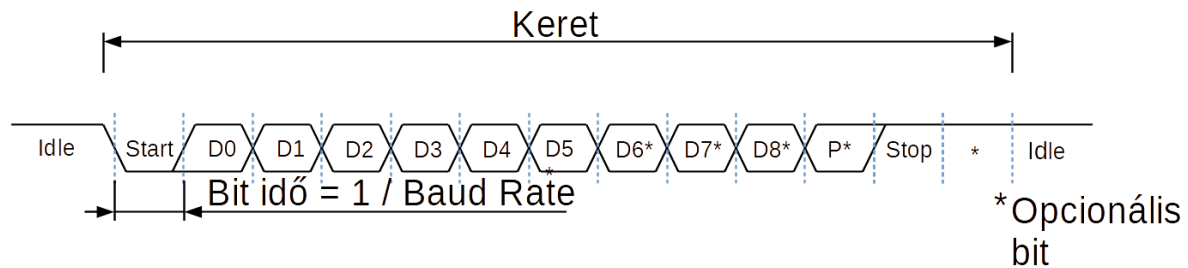
1. ábra: UART kommunikációs jelek megfelelő csatlakoztatása

Mikor UART kommunikációt említünk, erősen függ a felhasználás körülményeitől, hogy pontosan mire is gondolunk. Beágyazott eszközök esetén a modulok közötti kommunikációra általában pont-pont topológiát (két eszköz összekötve), TTL (tranzisztor-tranzisztor szintű) feszültségszinteket és pozitív logikát alkalmazunk. Az ipar különböző területein azonban más-más asszociáció él az emberekben. Ha egy rendszertervező szakembert kérdezzük meg a témában, valószínűleg valamelyik RS protokoll fog eszébe jutni (RS-232, RS-422, RS-485). Számunkra az úgynevezett UART-TTL variáns a releváns, mivel a mikroprogramozható eszközök jelentős része tartalmazza az ehhez szükséges perifériákat. Ez logikai 1 (mark) érték esetén 2,0-5,0 V-ot jelent, logikai 0 (space) esetében pedig 0-0,8 V-ot.

Kitekintés:

Sokat emlegettem már az RS protokollokat, most röviden szeretném bemutatni őket. Az RS betűszó a Recommended Standard (ajánlott szabvány) szavakból származik. Az ajánlást és a szabvány leírását az Electronic Industries Alliance (EIA) adta ki, és egészen a telekommunikáció hajnalára visszavezethető. Az idle (tétlen) állapot a feszültség alatt lévő állapot volt, melynek indoka a szakadás érzékelése, ami a szabványokban meg is maradt. Az RS-232 stabil kommunikáció elősegítéséhez Flow controlt alkalmaz, a 422 és 485 pedig differenciális jelpárokat használ (a két vezeték közötti feszültségkülönbség hordozza az információt), hogy nagyobb távolságot legyenek képesek áthidalni, a feldolgozás során így könnyebb kiszűrni a zajt.

Az UART nagy múltra tekint vissza, lényegében az egyik legrégebbi számítástechnikában használatos kommunikációs formáról beszélünk. Akkoriban 5 bites Baudot kódot használtak a karakterek átvitelére. Ebbe az angol ABC mind a 26 betűje és két speciális karakter is bőven belefért. Baudot-kódot feltalálójáról, Émile Baudotról nevezték el.



2. ábra: UART adatkeret

A fenti képen látható, hogy milyen módon történik meg egy adatkeret átküldése. Minden keretet (frame) startbittel kezdünk.

Általában 8 hasznos adatbitet küldünk egy keretben, de elterjedt az 5-9 bit is. Opcionális a paritás jelző bit jelenléte, ami a küldött adat bináris formáját egészíti ki páros vagy páratlan számú "1"-esre, így ellenőrizhetővé téve a beérkezett adat helyességét. Végül pedig a kommunikáció végét jelzi egy, másfél vagy kettő stopbit. Mindezt majd a példakódban be is mutatom.

Kitekintés:

A hasznos adat maximális (9 bit) mérete technológiai limitációk miatt alakult ki. Akkoriban nem volt könnyű olyan pontos órajellel működő rendszereket alkotni, mint ma. Mivel a kommunikáció folyamán a résztvevők a keretek elején szinkronizálják órajeleiket, így a biztonságos, szinkronizált kommunikációt maximum 10 bit (9 adat + paritás) átadására engedték meg. Ma már a kis hibahatárral rendelkező kristályokkal sokkal hosszabb adat küldését is képesek lennénk szinkronban tartani.

Másfél stopbit:

Hogyan létezhet másfél bites stop jelzés? Úgy, hogy a stopbiteknek az a feladata, hogy időt hagyjanak a vevő oldalnak az új adat fogadására való felkészülésre. A legtöbb esetben 1 stopbitet alkalmazunk, tehát legalább egy bit átvitelére szánt időt hagyunk ki két adat küldése között. A másfél bit esetén az a kikötés, hogy a kihagyott idő legalább másfél bit átküldéséhez szükséges idő legyen.

Ismerjük az adat küldésének formáját, nézzük meg, hogy mi kell még, ha két eszközt interakcióra akarunk bírni. Először is mindkét eszközön meg kell határozni a baud rate-et.

A baud rate az az arányszám, ami meghatározza, hogy hány darab szimbólum kerül továbbításra egy másodperc alatt. Most könnyű kiszámolni, hogy mennyi az annyi, hiszen UART kommunikáció esetén minden bit egy szimbólum, azaz egy 9 600-as baud rate, jelen esetben 9 600 bit/s-ot jelent.

Kitekintés:

Mivel a legtöbb UART eszköz magasabb órajelen működik, mint amit a kommunikációhoz használt baud rate megkövetel, ezért általában gyakrabban is mintavételezik a fogadott adatot (8-16-szorosan mintavételezik túl a jelet), mint azt feltétlenül szükséges volna. Így egy bit értékét több minta alapján tudja eldönteni az eszköz. Ez a mintavételezés első ránézésre roppant pazarló, hiszen jól időzítve egy mintából is tökéletesen meg lehetne állapítani az adat bit értékét. A kulcsszó ebben az esetben az időzítés. A két kommunikáló eszköz működési frekvenciája között a legtöbb esetben tapasztalható egy kis mennyiségű differencia. Ez a különbség okozhat hibát, ha

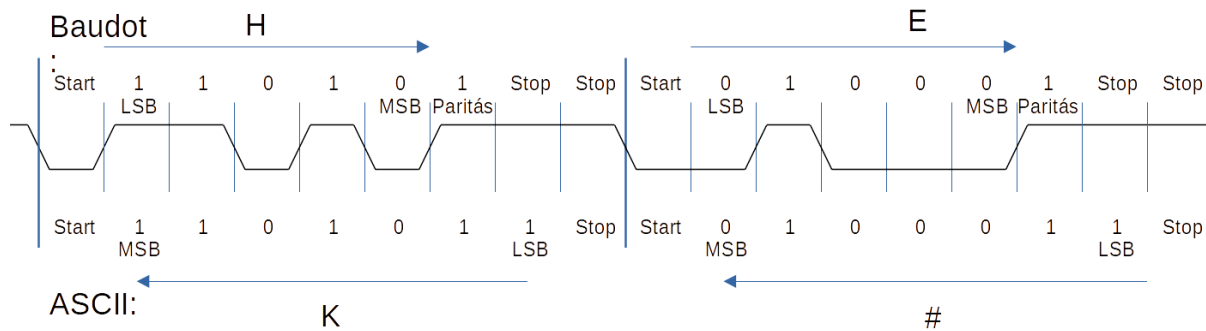
a kommunikáció hosszan tart, ezt hivatott kiküszöbölni a túlmintavételezés. Minél nagyobb mértékben mintavételezzük az adott jelet, annál nagyobb frekvencia-hibahatárral tud működni a kommunikáció.

Az auto baud rate fogalma ezen elv továbbgondolásából született. Eszerint a kommunikáció megkezdésénél az első küldött adat (ez általában egy egyezményes szimbólum például: "A" vagy "a", az esetek nagy részében ASCII kódolással) alapján a fogadó fél megfigyeli, hogy milyen paraméterekkel kell dolgoznia, vagyis milyen paritásbit, stopbit és bitráta szerint érkezik az adat. A bitsebesség azaz a bitráta az a sebesség, ami megadja, hogy másodpercenként hány darab bit fog átmenni a kommunikációs vonalon (mértékegysége a bit/s, vagy Hz). A későbbi adatoknál ezen beállítások szerint várja az információt. Jó példa erre, hogy az ethernet kommunikációban (hiszen ez is aszinkron módon működik) használt modem (jelmoduláló és demoduláló eszköz, elsősorban az internettel kapcsolatban találkozhatunk a fogalommal) képes ezt kezelni, illetve több aszinkron szabvány is van, aminek alapkövét képezi az auto baud rate detection.

Vannak elterjedt, gyakori baud rate beállítások az UART kommunikációhoz, de fontos megjegyezni, hogy – legalábbis elméletben – bármilyen érték megadható, ha azt képes teljesíteni az adott eszköz. A szabványosított készülékek esetén az órajel megfelelő osztása érdekében előre meghatározott értékek állíthatók be. A telekommunikáció hajnalán 75 bit/s-os sebesség volt népszerű, ma már inkább 300-600 bit/s-ot tartunk a skála aljának, de az egyik elterjedt alapbeállítás a 9600 bit/s. Maximum tekintetében határ a csillagos ég, azonban a teljesség igénye nélkül szeretnék felsorolni pár megszokott értéket: 14400, 19200, 38400, 57600, 115200, 230400, 460800, 921600 bit/s. Ne felejtjük el, hogy UART esetében a bitsebesség és baud-ráta értéke megegyező, de ez nem mindig van így!

Miután azonos a küldő és fogadó paraméterezése, gondolhatnánk, hogy minden rendben, mehet a beszélgetés! Nos, még nem. Tudnunk kell, hogy képes-e mindkét eszköz teljesíteni a kívánt paraméterekkel az időzíti kritériumokat. Más szóval tud-e a küldő olyan gyorsan adatot küldeni, mint azt a fogadó elvárja, vagy fordítva, tud-e a fogadó olyan sebességgel feldolgozni, mint ahogyan kapja az adatot? Elég az egy vagy másfél stopbit az adatok dekódolásához? Milyen környezetben tervezzük használni eszközünket? Szükséges-e paritásbit használata esetleges külső zavaró jelek (nagyfrekvenciás impulzusok / nagy áramok a közelben) által okozott hiba detektálása miatt? Sajnos ezekre a kérdésekre mind meg kell adni a választ, különben érdekes meglepetésekben lehet részünk. (Általában az eszközök csak olyan beállításokat engednek meg, amit képesek is kezelni, de mindez az adott eszköz adatlapjában biztosan megtalálható.)

Fontos kiemelni, hogy a paritásbit jelenléte nem elégséges hibaellenőrző módszer. Ha páros számú bit változik külső behatás miatt, akkor a paritás helyes eredményt ad, úgy, hogy a hiba jelen van. Ezen problémát gyakran úgy oldják meg, hogy a kommunikáció egy magasabb rétegében (a program feladata) beiktatnak minden n-edik keret után egy ellenőrző keretet (például: CRC - lásd később), amiben az addig elküldött adat valamilyen logika szerinti összege található.



3. ábra: Példa a különböző adat értelmezésre

A fenti képen látható két eltérő feldolgozása ugyanannak a jelnek. A felső értelmezés egy 5 bites Baudot-kód alapú hasznos adatot, egy páros paritásbitet és két stopbitet feltételez, a legalacsonyabb helyiértékű bitet küldve elsőként. Az alsó pedig egy 7 bites ASCII kódolást, paritásbit nélkül, egy stopbittel, az elsőként küldött bitet tekintve legmagasabb helyiértékűnek. Látható, hogy egyik dekódolás szerint “H” és “E” karaktereket kapunk, míg a másik szerint “K” és “#” szimbólumokat. Nagyon fontos a két kommunikáló eszköz között a feldolgozás metódus egységesítése. Ha ez nincs, akkor ilyen és hasonló problémákba futhatunk a kommunikáció során.

Mi történik akkor, ha minden szinten megvan a megállapodás, de valamilyen külső behatásnak köszönhetően mégis hibás adatot kap a fogadó fél? Ilyenkor hibajelzést kaphatunk az UART perifériától. Leggyakoribb a keret- és paritáshiba, előbbi a stopbitek hiánya vagy késése, utóbbit a hibás paritásbit okozza. Overrun (túllépés) hibát a fogadó oldal tud érzékelni, ha gyorsabban kap új startbitet, mint ahogyan az előző keretet feldolgozta a vezérlés. Underrun (hiányzó adat) visszajelzést várhatunk, ha nincsen küldendő adat, de kérés van a küldésre. Végül pedig egy különleges helyzet a break condition (megszakítási feltétel), ami hosszabb logikai 0 értékű jel (azaz alacsony feszültségszint) esetén következik be, ezt általában a vezeték szakadása okozza.

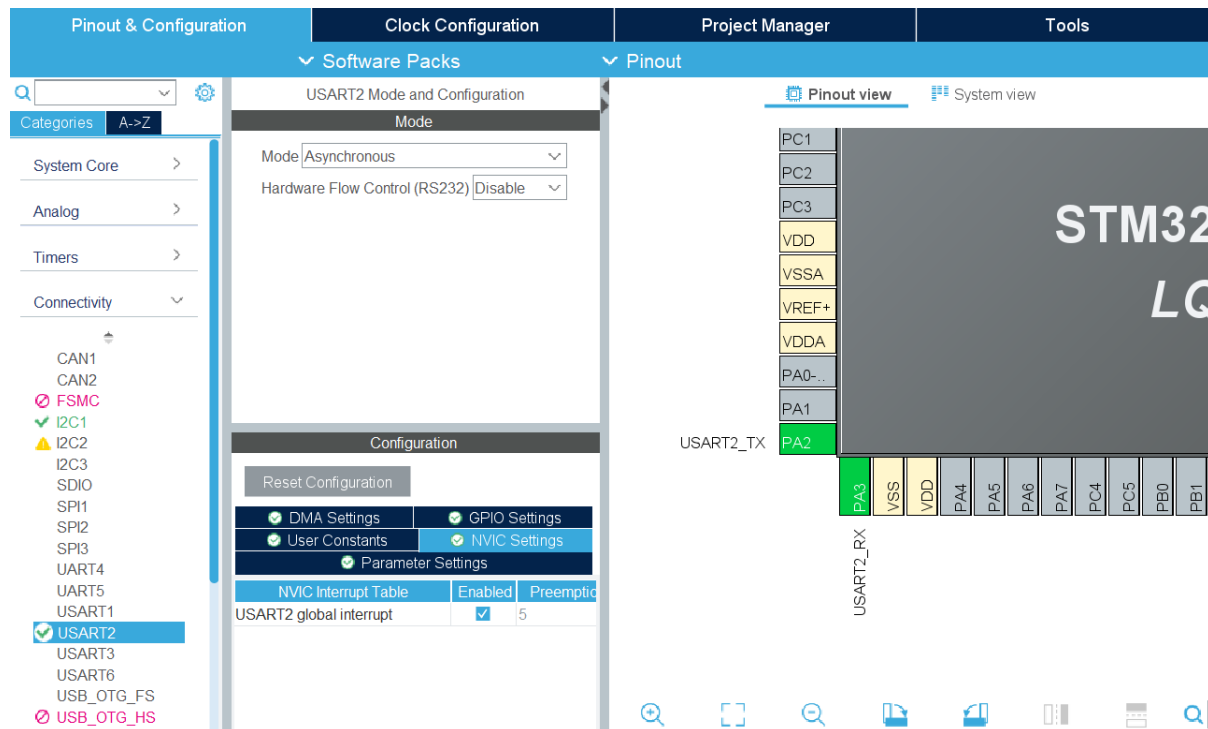
Na, most már ismerjük az aszinkron soros kommunikáció csínját-bínját. Nézzük meg, hogyan is néz ki ez a gyakorlatban! Amit eddig olvashattatok, az általánosan igaz a legtöbb UART perifériával megvalósított aszinkron soros kommunikációra. Ahogyan később a mikrokontroller ki- és bemeneteinek beállításainál is láthatjuk majd, rengeteg olyan kommunikációs protokoll van, ami az UART egy változatát alkalmazza. Az autópárházban ilyen a LIN (Local Interconnect Network) egyvezetékes protokoll, de még a telefonunkban megtalálható SIM (subscriber identity module, azaz előfizető azonosító modul) kártya is egy specifikus UART alapú kommunikációt használ.

UART PÉLDAPROGRAM

Az általunk használt STM32F4-es mikrokontrollercsalád tartalmaz (belső) UART perifériákat is (egészen pontosan kettőt), de implementáltak még benne 4 teljes USART-ot is (Universal synchronous and asynchronous receiver-transmitter - univerzális szinkron és aszinkron adó-vevő), ami az UART eszközök szinkron kommunikációra is képes rokona. Az USART perifériák tudásukat tekintve magasabb rendűek, mivel tudnak szinkron és aszinkron módon is működni, tehát tekinthetünk rájuk UART-ként és USRT-ként egyaránt, de akkor olyan módon kell konfigurálni őket.

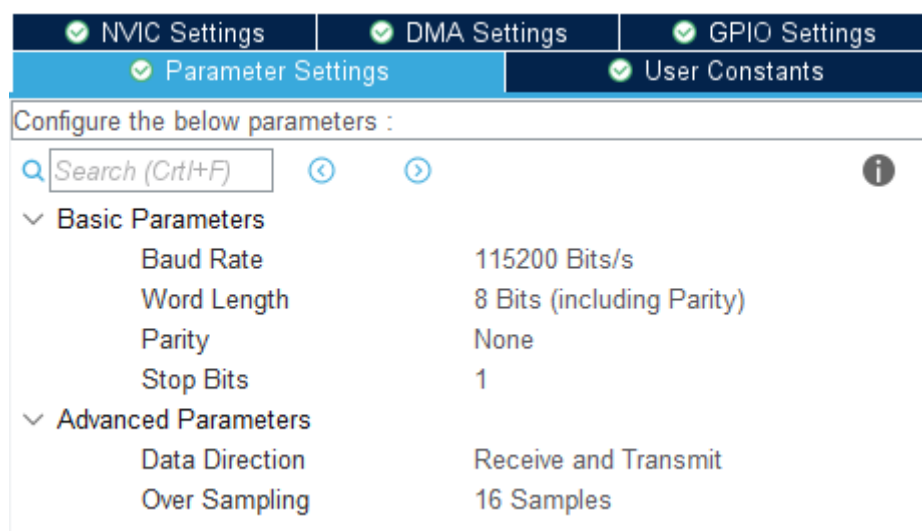
Importáljuk a “10_UART” példakódot a már megszokott módon, majd nyissuk meg a “.ioc” kiterjesztésű fájlt! Azt láthatjuk, hogy a legtöbb beállítással már találkoztunk az eddigi tananyagrészek

folyamán, azonban a mikrokontroller rajzának bal alsó sarokban kettő, eddig ismeretlen kivezetés van beállítva. A PA2-es és PA3-as lábakat USART perifériára konfiguráltuk.



4. ábra: USART csatorna kiválasztása

Kattintsunk a Connectivity menüpont alatt az USART2 feliratra, itt tudjuk beállítani, hogy milyen paraméterekkel szeretnénk használni a perifériát. Sokféle kommunikációs formát ismer a kontroller, melyeket a "MODE" legördülő menüben találhatunk. Jelen esetben (és gyakran általánosságban is) az Asynchronous lehetőséget válasszuk! Ezzel határozzuk meg, hogy az USART perifériánk UART-ként funkcionáljon. A korábban említett Flow Control-t pedig kikapcsoltuk.



5. ábra: USART csatorna paramétere

Lentebb a periféria paraméter beállításai között adhatjuk meg, hogy milyen baud rate-en, milyen hosszú adatot és milyen keretbeállításokkal szeretnénk küldeni. Ahogyan azt a képen is láthatjuk, a kommunikációnk 115200 bit/s sebességgel, 8 bit adattal, paritásbit nélkül és csak egy stopbittel fog

működni. Az általunk használt konfiguráció nagyon elterjedt, és általánosan 8N1 néven szoktunk hivatkozni rá. Fontos megjegyezni, hogy ha nincsen szükségünk kétirányú kommunikációra, akkor beállíthatjuk a perifériát csak küldő, vagy csak fogadó módon is. Az utolsó beállítás a túlmintavételezés mintaszámát határozza meg. Az STM32 mikrokontrollerek esetében alapbeállítás a 8 minta egy bitidőben, de ha az eszköz képes rá, célszerűbb a nagyobb túlmintavételezést használni, így mi is 16 mintára állítottuk ezt az értéket.

A példaprogramban egy olyan alkalmazást fogunk megismerni, ami egy pixelt irányítva rajzol a képernyőre. Az előző fejezet alapján ez nem is hangzik nehéznek, de minek ehhez az UART? Itt jön a turpisság, két összekötött mikrokontrolleren együtt lehet irányítani a kis pontot, tehát ha az egyiket megnyomjuk a fel gombot, a másik vezérlő képernyőjén is irányt fog váltani a pixel. Ehhez sajnos szükség van két mikrokontrollerre, de ha nálad csak egy van, kicsit kevésbé látványos, azonban ugyanannyira szórakoztató módon megismerhető az UART működése.

A két fejlesztőpanelt az "UART_TX" és "UART_RX" kivezetések segítségével lehet összekötni. Az egyik panel TX lábát a másik RX lábára kell kötni és fordítva. Amennyiben egy eszközzel szeretnénk kipróbálni a programot, akkor a breadboard-on kössük össze a TX és RX kivezetéseket egy 10 kΩ-os ellenállással, így a modulunk TX lábán történő adást mi magunk fogjuk megkapni, ezáltal lényegében önmagunkkal beszélgetünk.

Kitekintés:

Előfordulhat a bekapcsolás folyamatában olyan állapot, amikor az egyik láb tápfeszültségen, a másik pedig földön van, ami miatt rövidzár alakulhat ki. Ezt hivatott kivédeni az ellenállás azzal, hogy megszabja, mekkora áram folyhat.

A perifériák beállítása után térjünk rá a kódra! Mint minden perifériát, az UART-ot is inicializálni kell használat előtt. A képen látható kódrészletet az "usart.c" fájl megfelelő soraiban találhatjuk meg. Talán mindenkinek ismerősek lehetnek a beállított értékek. Ezt a fájlt az IDE generálja a korábban beállított ".ioc" kiterjesztésű fájl alapján.

```
UART_HandleTypeDef huart2;           // Paramétereket tartalmazó struktúra

/* USART2 init function */

void MX_USART2_UART_Init(void)
{

    huart2.Instance = USART2;           // Csatorna
    huart2.Init.BaudRate = 115200;       // Sebesség
    huart2.Init.WordLength = UART_WORDLENGTH_8B; // 8 bit adat
    huart2.Init.StopBits = UART_STOPBITS_1; // 1 stopbit
    huart2.Init.Parity = UART_PARITY_NONE; // Nincs paritásbit
```

```

    huart2.Init.Mode = UART_MODE_TX_RX;           // Kétirányú kommunikáció
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE; // Flow control kikapcsolva
    huart2.Init.OverSampling = UART_OVERSAMPLING_16; // 16x-os túlmintavételezés
    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
        Error_Handler();
    }
}

```

A többi újdonságot a "main.c" tartalmazza. A kód első soraiban röviden definiáljuk a futáshoz szükséges értékeket, deklaráljuk és inicializáljuk a változókat, majd beolvassuk a gombok állapotát. A gombok által meghatározott irányt az "x_velocity" és "y_velocity" változóban tároljuk el, majd ezeket az adatokat átadjuk a küldeni kívánt "tx_data_byte" változónak is. Az adat küldését és fogadását megvalósító kódsorok az alábbi képen láthatóak.

```

HAL_UART_Transmit(&huart2, &tx_data_byte, 1, UART_TX_TIMEOUT); // Küldés
HAL_UART_Receive(&huart2, &rx_data_byte, 1, UART_RX_TIMEOUT);  // Fogadás

```

Végül már csak érvényre kell juttatni a kívánt és kapott változtatásokat. Minden futási ciklust a 3-as LED egyetlen villanása jelez. Ezt heartbeat (szívverés) jelnek nevezzük és olyan esetekben szokás használni, amikor fontos kritérium, hogy a program megakadása egyértelműen észlelhető. Megfigyelhető, hogy ha a kódot egyetlen fejlesztőkártyán alkalmazzuk (a saját TX-RX kivezetést összekapcsoló ellenállás nélkül), akkor lassabb a szívverés frekvenciája, mintha két készülék lenne összekötve. Ezen jelenség oka a HAL_UART_Receive() függvényben keresendő. Ennek fontos paramétere az UART_RX_TIMEOUT, amely értéke jelenleg 200. Ez határozza meg azt, hogy a függvény maximálisan mennyi ideig vár új beérkező adatra. Amennyiben (hamarabb) érkezik információ, akkor hamarabb befejeződik a függvény futása, viszont egy eszköz esetén csak kiolvassuk a regiszterben tárolt értéket, de nem kapunk új értéket, így timeout-olni fog a függvény. Érdemes megjegyezni, hogy a küldő függvény (HAL_UART_Transmit()) is rendelkezik ilyen paraméterrel (UART_TX_TIMEOUT). A küldő algoritmus tipikusan akkor várakozhat az adat elküldésére, ha valamilyen okból kifolyólag megszakad a program futása. Ekkor maximum a timeout idejéig vár a függvény, majd hibát jelezve tér vissza.

Egy vezérlő esetén azt tapasztalhatjuk, hogy minden kívánt folyamat kétszer történik meg, ezt a legkönnyebben a középső gomb megnyomásával figyelhetjük meg. Először a gombnyomás hatására történik értékmódosítás, majd pedig az UART-on (önmagától) kapott adat hatására.

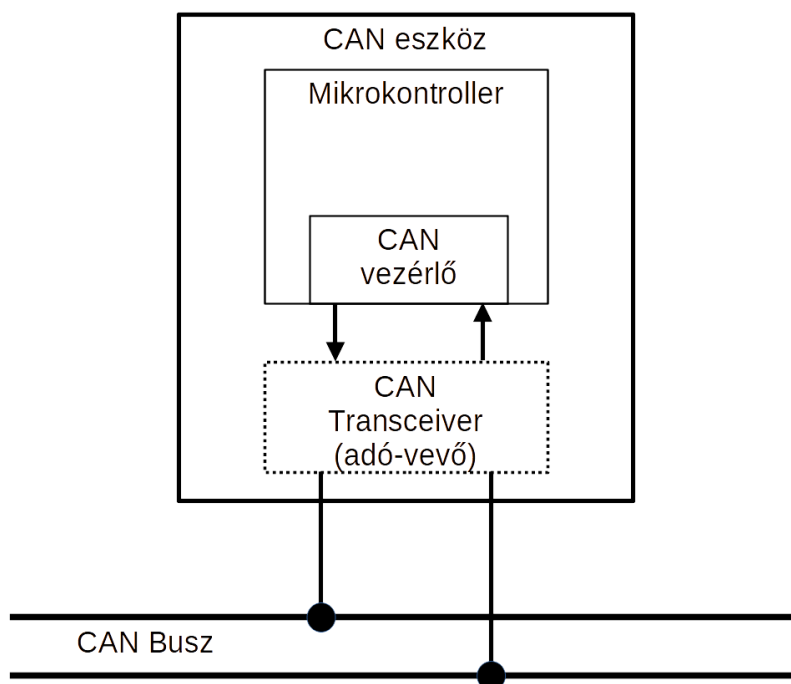
CAN

Az iparban, különösen a járműiparban egyfajta standardként van jelen a CAN (Controller Area Network). Ezt a szabványt kifejezetten autókhoz tervezték, a vezetékek számának redukálása céljából. 1986-ban mutatta be a Robert Bosch GmbH három év fejlesztés után. A legutóbbi teljes szabvány, a CAN 2.0 1991-ben látott napvilágot. A célkitűzés az volt, hogy az autóipar számára megbízható, olcsó és egyszerű kommunikációs rendszert fejlesszenek. Olyan sikeres lett a CAN, hogy ma már jelentős mértékben elterjedt a használat az autóiparon belül és kívül is (több, mint 60%-ban egyéb iparágak alkalmazzák, többek között egészségügyi eszközökben és egyéb nagy megbízhatóságot igénylő készülékekben).

Kitekintés:

A CAN 2.0 szabvány megjelenésének évében mutatták be a Mercedes-Benz W140-t is, ami az első olyan jármű, amiben CAN-busz alapú kommunikációt használtak.

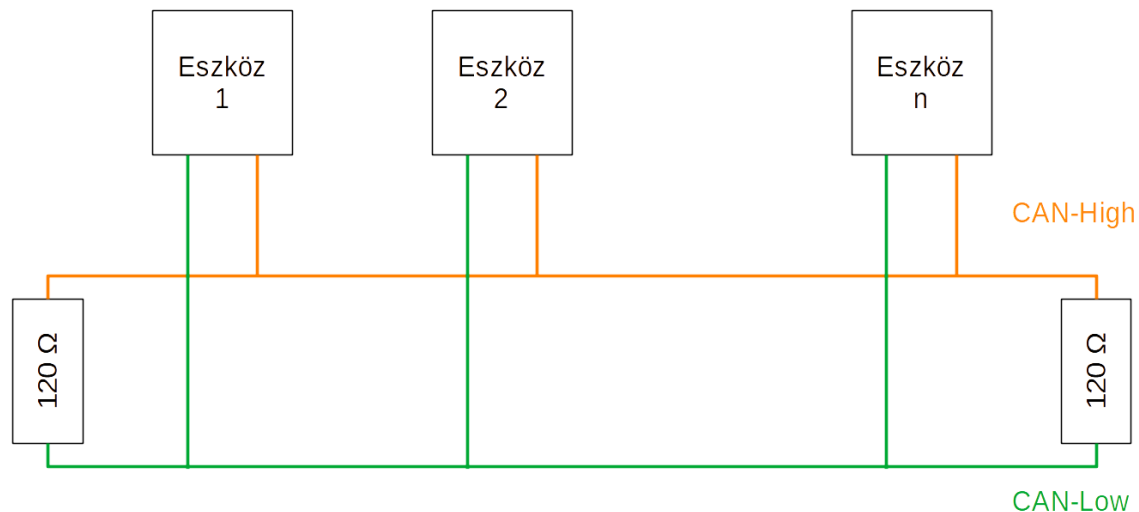
A CAN egy multimaster protokoll, tehát ellentétben az I²C és SPI általános felhasználásával (mindkettő esetben megoldható a multimaster topológia, azonban ritkán alkalmazzák) nem egyetlen master eszköz és több slave eszköz játszik szerepet, hanem minden eszköz egyenlő rangú. Ezeket a készülékeket node-oknak (vagy ECU-knak, Electrical Control Unit-nak) nevezzük. Minden node-nak van egy vezérlő egysége (mikrokontroller), CAN vezérlője és CAN adó-vevő egysége.



6. ábra: CAN kommunikáció fizikai felépítése

A mikrokontroller feldolgozza a kapott adatot, vagy előkészíti a küldendő adatot. A CAN controller átalakítóként szerepel a CAN logika és a mikrokontroller logikája között, végül a CAN adó-vevő a fizikai rétegbeli átalakítást végzi el. Ez utóbbi tartalmaz kiegészítésként különböző védelmeket (például: túláram és túlfeszültség ellen).

A node-ok üzenetei között prioritás alapú hierarchia van. Minden üzenetnek van egy egyéni 11 (vagy 29) bites azonosítója, melyek között a 0-s érték a legmagasabb prioritású (Később is elő fog fordulni, hogy az alacsonyabb érték jelenti a magasabb prioritást). Hogy miért a 0? Ehhez meg kell érteni a fizikai összeköttetés működését.



7. ábra: CAN összeköttetés fizikai megvalósítása

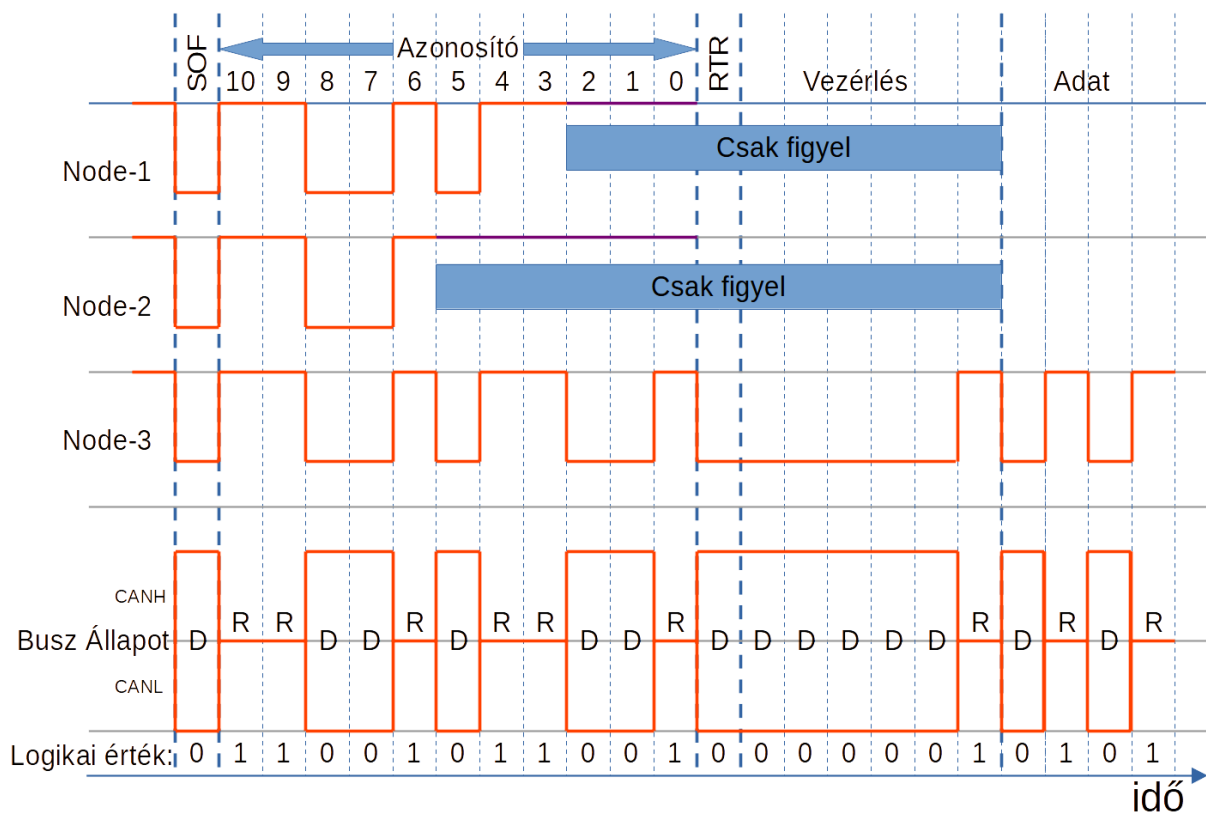
A CAN fizikai megvalósítás tekintetében két összecsavart vezetékert jelent (csavart érpár), amelynek a 2 végpontjában található eszközöknél mindig egy-egy 120 Ω -os lezáró ellenállás szerepel. Ez a két vezeték egy úgynevezett differenciális jelátvitelt valósít meg. A feszültségeket, amiknek különbsége határozza meg a jel értékét a CANL (CAN Low) és CANH (CAN High) vezetékekből álló csavart érpáron mérhetjük. Szemben a korábban megismert magas és alacsony jelöléssel a CAN protokoll domináns (logikai 0) és recesszív (logikai 1) értékeket különböztet meg. Amennyiben $CANH > CANL$ akkor domináns érték, ha $CANH \leq CANL$ akkor recesszív érték jelenik meg a buszon (természetesen nem ilyen egzakt módon működik, sok tűrés van belekalkulálva).

A CAN busz úgynevezett negált logikát követ. Tehát, amikor CANH és CANL vezetékek között mérhető feszültség (amely a protokollban specifikált tartományba esik), akkor ez logikailag egy 0 bitnek felel meg; míg elengedett busz esetében, ha CANH és CANL ekvipotenciálisak, tehát a köztük mérhető potenciálkülönbség 0, akkor ez logikai 1-es bitet jelent.

Például, ha $CANH = 5\text{ V}$ és $CANL = 0\text{ V}$ akkor $U_{CANH} - U_{CANL} = 5\text{ V}$, míg, ha $CANH = 2,5\text{ V}$ és $CANL = 2,5\text{ V}$, akkor $U_{CANH} - U_{CANL} = 0\text{ V}$, tehát sorrendben 0 és 1 a logikai érték.

Az elnevezés onnan származik, hogy ha egy node recesszív jelet küld, egy másik pedig dominánsat, akkor az utóbbi fog megjelenni a buszon. Úgy van kialakítva a fizikai rétege a CAN protokollnak, hogy a differenciális vezetékpár és kapcsolatot valósít meg, hiszen a domináns jel esetén a két vezetékert "széthúzzuk", míg recesszív esetén "elengedjük". Így, ha minden node elengedi a buszt, akkor az azon található feszültségkülönbség kiegyenlítődik a két lezáróellenálláson keresztül, de ha akár csak 1 node is széthúzza, akkor nincs mód a kiegyenlítésre. Ha nincsen meghajtás a buszon, akkor a CANH és CANL vezetékek potenciálja közel van egymáshoz ($CANH \leq CANL$) vagyis recesszív állapotban van a busz a viszonylag kis ellenállás miatt. A prioritás is ide köthető: az alacsonyabb értékű azonosítóban kisebb helyiértéken szerepel 1-es, tehát az fog kijutni a buszra. Mivel azonos címek nem fordulhatnak elő, így nem alakul ki ütközés a kommunikáció során. Hasonló megfontolásból a tétlen állapotot is a recesszív állapot jelzi. Az említett viselkedésből származik az a tulajdonság is, hogy ha két eszköz egyszerre kezd

el adatot küldeni a buszra, akkor a magasabb prioritású töretlenül folytatja, míg az alacsonyabb észleli az ütközést és leáll az adással.



8. ábra: Három node azonos időben történő kommunikációja (Versenyhelyzet / Arbitráció)

A fenti képen látható egy olyan szituáció, amikor versenyhelyzet alakul ki a buszon. A három node egyszerre kezdi meg az adást, és egészen az 5. azonosító bitig együtt is folytatják, azonban a kettes node magasabb ID-vel rendelkezik, így amint észleli, hogy nem az általa meghatározott adat jelenik meg a buszon, leáll az adással, és figyelőként folytatja működését. Hasonló jelenet játszódik le a 2-es ID bitnél, itt az első node észleli a változást és teszi meg a szükséges lépéseket.

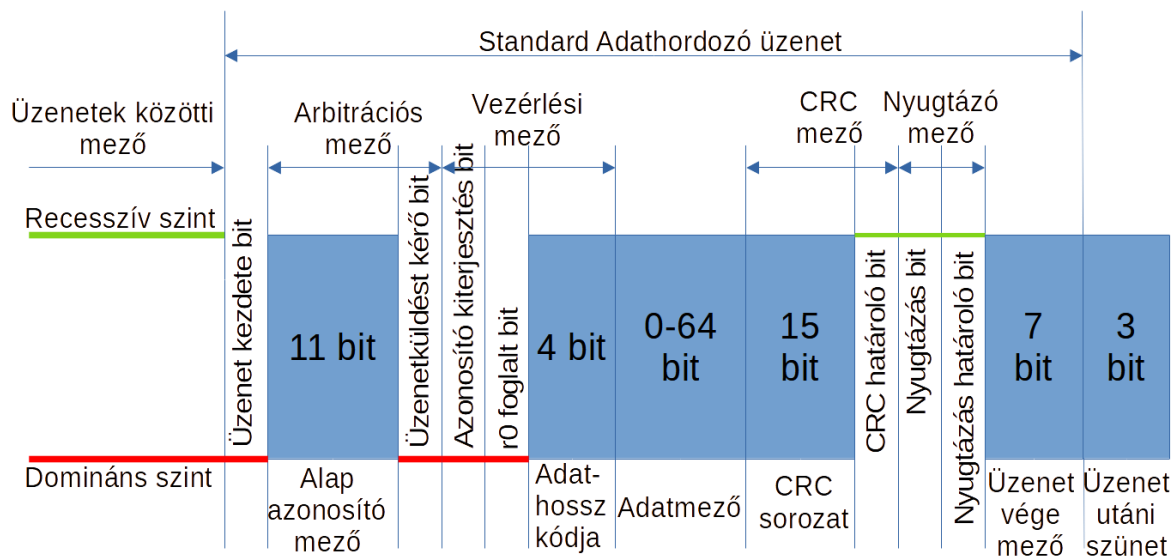
A kommunikáció szinkronizált, de aszinkron működésű. Tudom, hogy ez félreérthető, de rövidesen megmagyarázom. A protokoll alapul veszi azt, hogy minden node a megfelelő időben mintavételezi a buszt. Ezt úgy tudják elérni, hogy a kommunikáció megkezdésénél van egy egyeztetés az első recesszív - domináns átmenetnél (startbit), majd minden egyes

logikai 0-1 átmenetnél megfigyelik a résztvevők, hogy az a standard bit idő hányszorosa alatt megy végbe. Így, habár a kommunikáció szinkronizált működésű, mivel nem rendelkezik órajellel vagy egyéb szinkronizálást biztosító jellel, a szinkronitás definíciója szerint aszinkronnak minősül.

A CAN szabvány szerint a maximális sebesség 1 Mbit/s, amit biztosítani tud a busz, azonban vannak típusok, amik 5 Mbit/s vagy 125 kbit/s maximális sebességekkel operálnak. Mindezt a megfelelő lezárással (lezáró 120 Ohm) és a meghatározott legnagyobb távolságon belül képes biztosítani a szabvány.

A fizikai réteget már értjük, jöjjön az adatkapcsolati réteg felépítése! Csakúgy, mint korábban, itt is keretek (frames) adják a kommunikáció alapját. A CAN 2.0 szabványon belül két különböző típust

különböztetünk meg, CAN 2.0A az alap verzió, (ami kisebb módosításokkal megfeleltethető a CAN 1.0-nak,) és a CAN 2.0B a kibővített kiadás. A kettő között mindössze az ID 11 bitről 29 bitre való kibővített hossza a különbség. Az alábbi képen az egyszerű 2.0A adatkeret látható.



9. ábra: CAN 2.0A Adatkeret (Wiki)

A CAN protokoll 4 különböző keretípust különböztet meg:

- Adatkeret (Data Frame)
- Hibakeret (Error Frame)
- Távoli keret (Remote Frame)
- Túlcscordulás keret (Overflow Frame)

Ahogy az a képen is látható, az **adatkeretek** "start of frame" bittel kezdenek, ami után az üzenet azonosítója és távoli adatkérés bitje következik. Az azonosítót és az adatkérés bitet szoktuk együttesen arbitrációs mezőnek nevezni, a különböző üzenetek közötti prioritás alapján itt dől el, hogy melyik nyer egy versenyhelyzet kialakulásakor. Két vezérlőbitet követően (bővített ID-t jelző bit és egy lefoglalt bit a szabvány továbbfejlesztéséhez) az adat hosszát meghatározó 4 bit következik (DLC = Data Length Code), majd az adat (DB = Data Bits), ez 0 és 8 byte közötti mennyiség lehet, ami a legmagasabb helyiértékű bittel kezdődik. A beépített CRC (Cyclic Redundancy Check) ellenőrzés a következő 16 bit, amiből 15 bit ellenőrzőösszeg és 1 bit lezáró 1-es. Az adatellenőrzés végén található egy olyan érték, amit nem a küldő fél állít, hanem az adást vevő eszközök jeleznek vissza vele. Amennyiben hiba lép fel a visszajelző bitnél vagy máshol, és úgy van beállítva az eszköz, akkor a küldő fél megismétli az adást. Végül pedig 7 egymást követő egyes jelenti a keret lezárását. Két keret között mindig legalább 3 bitidőnyi egyesnek kell jelen lennie.

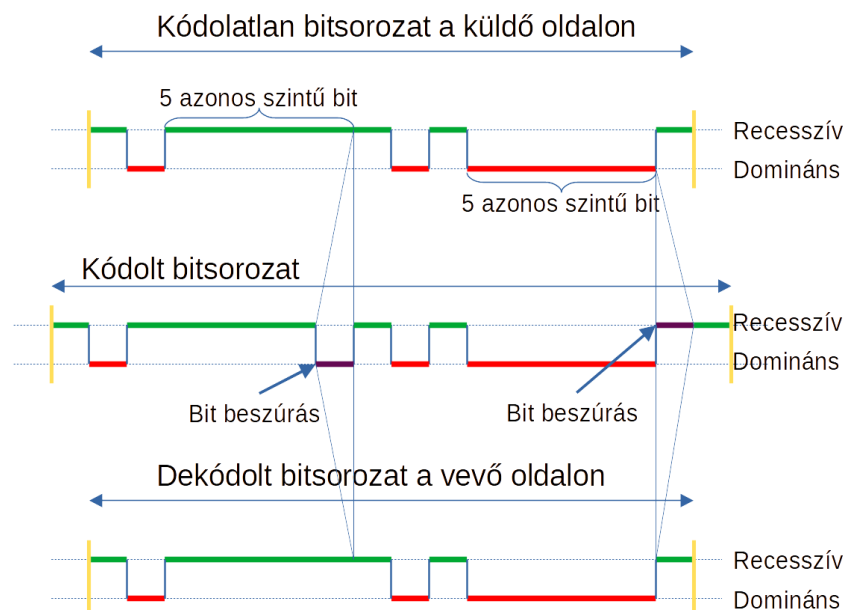
A **hibakeret** ennél egyszerűbb és rövidebb. Ha bármelyik node hibát fedez fel, akkor azonnal elkezd adni a hibakeretet, ami két részből tevődik össze: az Error Flag, ami 6 azonos, beállítástól függően 0 vagy 1 értékű bit, amit az Error Delimiter követ 8 db egyessel. A később bemutatott bitbeszúrási módszer miatt a 6 azonos értékű bit egyértelműen jelzi a hibát.

A **távoli keret** nagyon hasonló felépítésében az adatkerethez. A távoli adatkérés bit dominánsan szerepel, valamint a távoli keretben nincsen adatmező, vagyis ezen keret küldése esetén adatot várunk vissza.

A **túlcordulás keret** lényegében a hibakeret egy verziója 6 domináns és 8 recesszív bittel. Két situációban fordulhat elő: az egyik, mikor a vevő több időt kér az előző adat feldolgozásához, a másik pedig, ha valamelyik node megsérti a keretek közötti időkritériumot.

BITBESZÚRÁS:

Mivel nincsen külön órajel, a node-ok a recesszív-domináns átmenetek alapján állítják be a saját órajelüket. A gyakori szinkronizálás fontossága miatt bitbeszúrás, avagy bit beékelést alkalmazunk. Ez azt jelenti, hogy ha a buszon 5 azonos értékű bit jelenik meg, akkor az adó beszúr egy ellentétes értékű bitet. Ezeket a beszúrt biteket a vevő oldal felismeri - hiszen a beszúrt bitekkel nő az üzenet hossza -, és az üzenet kiértékelésénél nem veszi figyelembe, kvázi eldobja. Természetesen, ha hiba van a beszúrásban, azt észleli az eszköz.



10. ábra: Bitbeszúrás a CAN buszon

Ennek két óriási előnye is van: az egyik a már korábban említett legfeljebb 5 bitidőnkénti szinkronizálási lehetőség, a másik pedig a hibakeretek terjedésének lehetősége. Amennyiben valamelyik vevő hibát észlel, 6 azonos értékű bitet ad ki, így a buszon jelen lévő többi node is hibát jelez, így biztosítva, hogy minden node észleli a hibát.

Most már elméletben értjük, hogy is működik a CAN, de hol találkozhatunk vele a gyakorlatban? Sajnos a hobbielektronika nem nagyon alkalmaz CAN alapú megoldásokat, de az ipar annál inkább. Ahogyan azt már korábban is említettem, az autóipar számára lett kifejlesztve, ami azóta is rendületlenül alkalmazza. A mai autók szinte kivétel nélkül a különböző részegységvezérlők és szenzorok kommunikációját CAN-buszon valósítja meg. Például gondoljunk a gázadás folyamatára egy elektromos autó esetén! A gázpedálszenzor adata CAN-buszon keresztül jut el a motorvezérlőhöz, ahonnan ugyanezen a buszon keresztül jut el a sebesség-, fordulatszám- és hőmérséklet-információ a műszerfalra. (Arról már nem is beszélve, hogy némelyik autórádió a sebesség alapján felhangosítja vagy lehalkítja a zenét, amihez az információt a CAN-busztól nyeri.)

USB

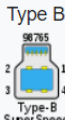
Az USB (Universal Serial Bus) szabvány a listánk talán legkülönlegesebb darabja. Ellenben az eddig bemutatott kommunikációs formákkal, az USB protokoll nemcsak a kommunikáció fizikai- és adatkapcsolati rétegeit definiálja, hanem a csatlakozók és kábelek teljes specifikációja mellett a tápellátással kapcsolatos kérdésekben sem hagy semmit a képzeletünkre.

A szabványt 1994-ben kezdte el fejleszteni hét cég: Compaq, DEC, IBM, Intel, Microsoft, NEC, és Nortel. A szabvány tervezésének célja az volt, hogy a számítógépekhez használt sok különböző csatlakozót lecseréljék egy univerzálisan alkalmazhatóra. 1996-ban jelent meg hivatalosan az USB 1.0, ami a már említett formai követelmények egyszerűsítésén túl az adat formátumát és az átviteli sebességet is javította. A különböző bonyolultságú rendszerek támogatása érdekében két sebességprofilot hoztak létre. Az egyik a Low Speed (alacsony sebesség), ami elméleti maximum szerint 1,5 Mbit/s-os sebességgel közvetíti az adatot, a másik pedig a Full Speed (teljes sebesség), ahol ez az érték 12 Mbit/s. Az előbbi elsősorban HMI (Human-Machine Interface) eszközökhöz, egerekhez, billentyűzetekhez tervezték, míg az utóbbit az akkori nagy sebességigényű perifériákhoz, mint a floppylemez olvasók vagy nyomtatók.

Az USB 2.0 2001 végén jelent meg, és megnegyvenszerezte elődje sebességét a High Speed (nagy sebességű) móddal, 480 Mbit/s-ra. Az ezt követő 3.0, 3.1 és 3.2-es szabványverziók mind kiegészítették a funkcionalitást, és növelték a sebességet 5-10-20 Gbit/s-ra. A tananyag írásának pillanatában az USB4 a legfrissebb verzió, amely 40 Gbit/s elméleti maximumot jegyez.

Minden USB eszköznek van egy Forgalmazói (Vendor ID) és Termék (Product ID) azonosítója. Ezen paraméterek alapján tudja meghatározni a számítógép, hogy milyen driver-t alkalmazzon a kommunikáció helyes működéséhez. Ezeket az ID-kat az USB-IF (USB Implementers Forum) tagjai kérvényezhetik és feltétlenül szükségesek az eszközök működéséhez. Ez egy nonprofit szervezet, melynek fő tevékenysége az USB népszerűsítése.

A rövid történelmi áttekintés után térjünk rá a csatlakozókra! Az első, USB 1.0-s szabvánnyal két csatlakozótípus került bemutatásra: a Type-A és Type-B, majd a második revízióban (USB 2.0) megjelentek a Mini-A és Mini-B csatlakozók. Azért létezik kettő fajta csatlakozó, mivel az USB master-slave kommunikáció és a csatlakozótípusok is erre referálnak. Ezért van az, hogy a számítógépeken USB Type-A, míg a nyomtatókat USB Type-B csatlakozókkal szerelték fel, még a könyv írásának időszakában is, mert így nem történhet meg az, hogy két hosztot/master-t (például számítógép, laptop) véletlenül összekötünk. Az alábbi hivatkozáson látható az összes említett típus és kompatibilitásuk a szabványokkal.

Connectors		USB 1.0 1996	USB 1.1 1998	USB 2.0 2001	USB 2.0 Revised	USB 3.0 2011	USB 3.1 2014	USB 3.2 2017	USB4 2019
Data rate		1.5 Mbit/s (<i>Low Speed</i>) 12 Mbit/s (<i>Full Speed</i>)		1.5 Mbit/s (<i>Low Speed</i>) 12 Mbit/s (<i>Full Speed</i>) 480 Mbit/s (<i>High Speed</i>)		5 Gbit/s (<i>SuperSpeed</i>)	10 Gbit/s (<i>SuperSpeed+</i>)	20 Gbit/s (<i>SuperSpeed+</i>)	40 Gbit/s (<i>SuperSpeed+</i> , <i>Thunderbolt 3</i> <i>and 4</i>)
Standard	A	Type A 				Type A 		Deprecated	
	B	Type B 				Type B 		Deprecated	
	C	N/A		Type C (<i>enlarged</i>) 					
Mini	A	N/A	Mini A 		Deprecated				
	B		Mini B 						
	AB	N/A	Mini AB 						
Micro	A	N/A	Micro A 		Micro A 		Deprecated		
	B	N/A	Micro B 		Micro B 		Deprecated		
	AB	N/A	Micro AB 		Micro AB 		Deprecated		
Connectors		USB 1.0 1996	USB 1.1 1998	USB 2.0 2001	USB 2.0 Revised	USB 3.0 2011	USB 3.1 2014	USB 3.2 2017	USB4 2019

11. ábra: Különböző USB csatlakozók és hozzájuk kapcsolódó szabványok

Forrás: Wikipédia

[https://en.wikipedia.org/wiki/USB#Receptacle_\(socket\)_identification](https://en.wikipedia.org/wiki/USB#Receptacle_(socket)_identification)

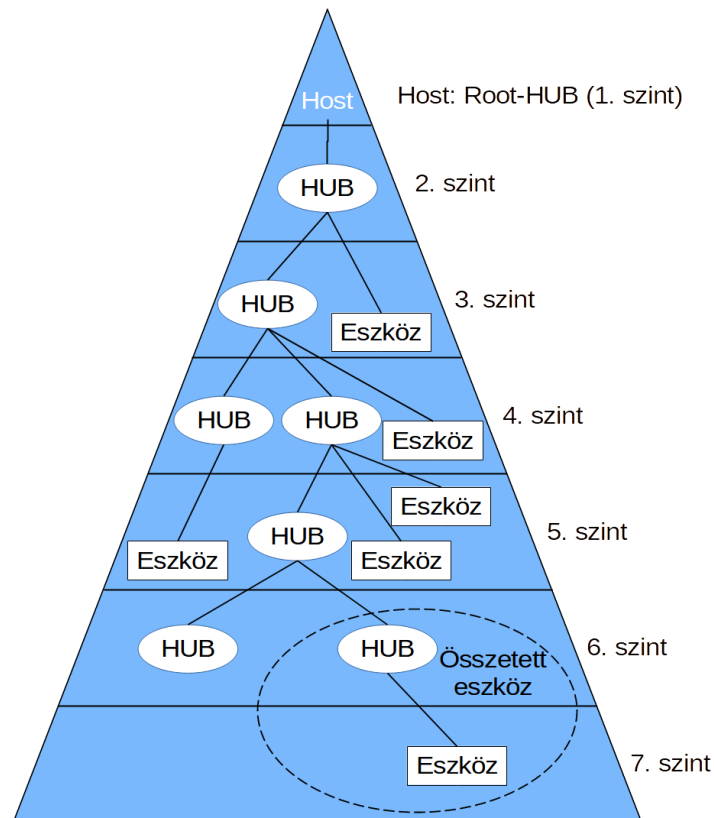
A ma már jól ismert Micro-B, valamint a kevésbé közismert Mini-AB és Micro-AB 2007-ben, az OTG-vel (On-The-Go) együtt jelent meg. Az USB 3.0 két teljesen új csatlakozót mutatott be, melyek szintén Type-A és Type-B néven ismeretesek, általában kék színük miatt ismerhetők fel. A Micro vonalat egy Micro-B SuperSpeed nevű konstrukcióval bővítették, ezt leggyakrabban külső merevlemezekben

láthatjuk. A Mini és Micro méretű csatlakozók az USB C típusú csatlakozó 2014-es elterjedésével visszaszorultak az évek alatt.

Már az USB 1.0-s csatlakozók is tartalmazták a föld- és tápvonalakat, habár akkor még más áramokra voltak minősítve a kábelek és csatlakozók. A meghatározott feszültség az USB Power Delivery (PD) szabvány megjelenése előtt 5 V volt 5%-os tűréssel. A rendszerben minden alacsony fogyasztású (low-power) és még nem inicializált nagy fogyasztású (high-power) eszköz maximum 1 egység áramot fogyaszthat. Normál üzemben a nagy fogyasztású eszközök minimum 1 és maximum 5 egységet. Az USB 3.0 előtt 1 egység 100 mA volt, utána pedig 150 mA (ez az általános eszközökre vonatkozik). Az akkumulátorok töltésére külön, a találón Battery Charging-nak (BC) elnevezett specifikáció alkalmazható, ami szerint ennek fogyasztása maximálisan 5 A lehet. Elérkezve 2012 nyarához megjelent a PD szabvány, és vele a magasabb feszültségszintek is. Az új technológia 5, 12 és 20 voltos rendszereket tudott meghajtani. Ezt később a második kiadás toldotta meg egy 9 voltos opcióval is. A tananyag írásának pillanatában 100W a maximálisan leadható teljesítmény egy (erre felkészített) USB kábelben keresztül.

A szabványok kiterjednek továbbá a csatlakozókra és a kábelekre is.

A szabvány adatkapcsolati rétegéről nem fogunk részletesen beszámolni, mert az univerzális felhasználhatóság miatt sokkal komplexebb a felépítése, mint például az SPI-é, így messze túlmutat ezen tananyag rész keretein. Érdeemes tudni azonban, hogy egy busz típusú csatornáról van szó, amelyen az általános szabvány szerint egy host (master) és akár 127 device (slave) helyezkedhet el. Több szintű csillagtopológiával rendelkezik a protokoll, és minden kommunikációs folyamat kezdeményezője a host eszköz. Hub (elosztók) segítségével több eszközt is lehet csatlakoztatni, ilyenkor minden hub alatti eszköz kommunikációja áthalad a hub-on.



12. ábra: Többszintű csillag topológia az USB esetében

Minden eszköz rendelkezik egy device class ID-vel (eszközosztály-azonosító), ami meghatározza, hogy milyen drivert használjon a host az adatok értelmezéséhez. Amikor csatlakoztatunk egy device eszközt egy host eszközhöz, megtörténik az azonosító kiküldése, és a host eldönti, hogy milyen prioritással szolgálja ki a device-t. Ezt a folyamatot nevezzük enumerációnak. (Irodalomból ismerős lehet a kifejezés, az eposzi kellékek között szerepel, jelentése: seregszemle.)

A differenciális jelpáron a kommunikáció itt is keretekben történik, azonban itt a keretek időbeli hossza maximalizálva van 1ms-ra. Egy keretben nem csak egy adatcsomag kerül átadásra, hanem a keret kezdete és vége között fel van osztva prioritás szerint az idő. A host minden device-nak allokal egy időablakot, amelyet az vagy kitölt, vagy nem.

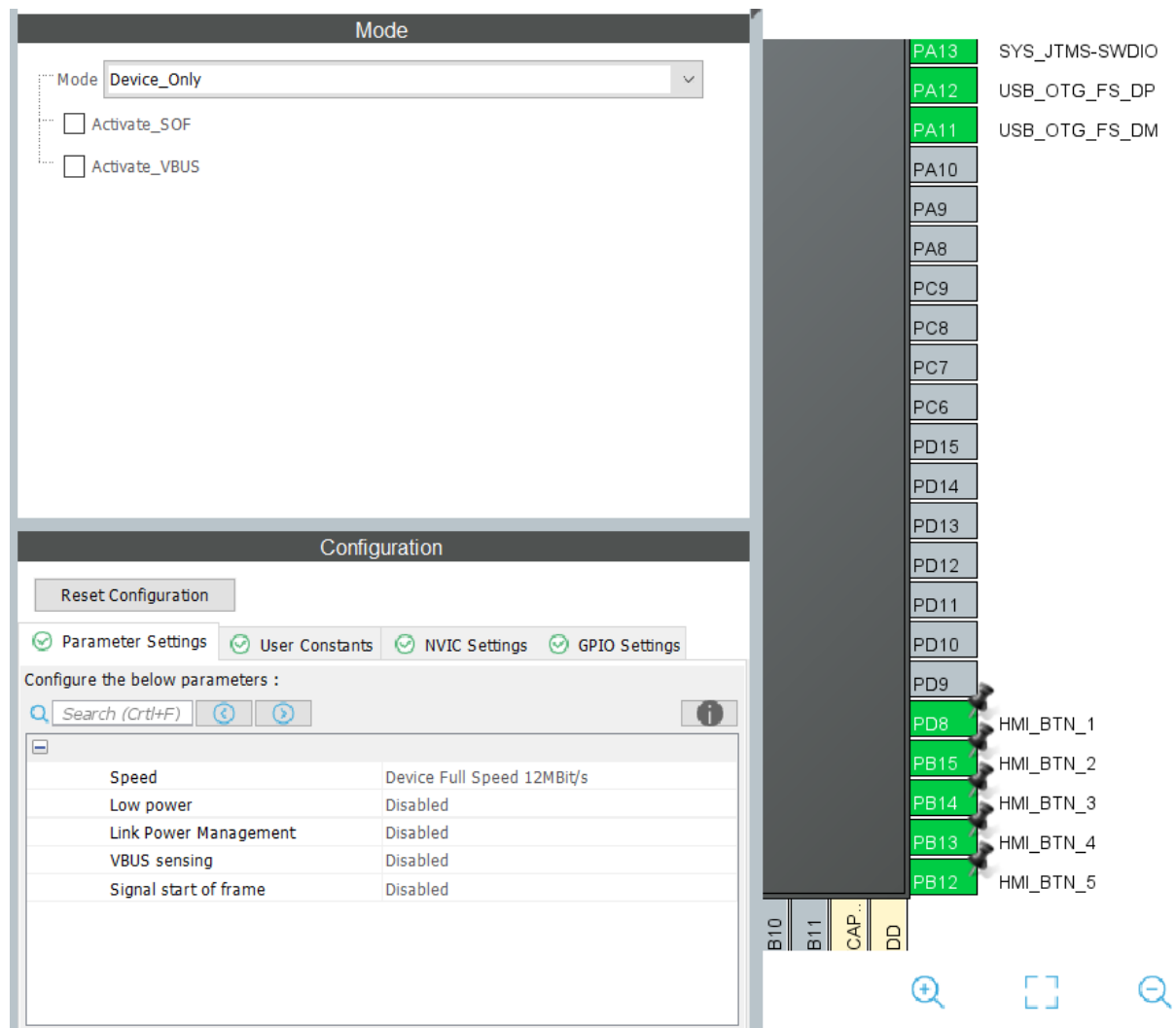
Az USB-s eszközök lehetnek csak host-ok vagy device-ok, de a 2007-es USB On-The-Go (OTG) specifikálja a lehetőséget, hogy ugyanabban a rendszerben szerepet cseréljenek az eszközök. Azt is fontos kiemelni, hogy egy fizikai eszköz a rendszer szempontjából lehet akár több külön feladatot ellátó egység is, egy rejtett hub-bal összekapcsolva. Tökéletes példa erre egy mikrofonos webkamera, ahol a hangot és képet egymástól szeparált módon is képes kezelni a számítógépünk. Ezeket a device-okat hívjuk compound device-oknak (összetett eszközök).

Manapság már nehéz úgy élni, hogy nem ismerjük az USB protokoll által nyújtott lehetőségeket, de szeretném kiemelni, hogy mennyire is univerzális ez a soros busz. Vegyük egyetlen USB portját a számítógépünknek! Szerintem lenyűgöző, hogy ezzel az egy csatlakozóval összeköthetünk tölteni kívánt eszközt, videó- és audioeszközöket, adattárolásra használt készülékeket, vagy az újabb szabványok szerint akár kijelzőket is. Ezt az egy csatlakozót használhatjuk az egér-, billentyűzet-, fejhallgató-, vagy mikrofonbemenetként is.

USB PÉLDAPROGRAM

Szerencsére a fejlesztőpanelen lévő mikrokontroller rendelkezik USB perifériával, ráadásul USB PHY-al (PHYsical layer = fizikai réteg) is, ami azt jelenti, hogy nemcsak az logikai tartalmát tudja előállítani, hanem közvetlenül a jelszinteket is. Tehát nincsen szükség a Kristálytisza Elektronika tananyagban megismert átalakító áramkörre. A szabvány komplexitása miatt inkább érdekességként szeretnénk bemutatni az USB periféria HID eszközként való felhasználását. A Human Interface Device-ok azok a perifériák, amelyek segítségével az ember interakcióba lép a számítógéppel. Ilyen például az egér, billentyűzet, joystick stb. A példakódban egy egeret fogunk készíteni, amit a négy szélső gomb segítségével tudunk majd irányítani és a középsővel kattintani. A program erősen épít az ST által kiadott USB stack-re, ez azt jelenti, hogy a pontos regiszter szintű működést nem fogjuk látni, mindössze egy magasabb absztrakciós szinten fogunk dolgozni (csakúgy, mint tettük azt az 5. tananyagrészen).

Importáljuk a "10_USB" példakódot, majd nyissuk meg a .ioc kiterjesztésű fájlt! Ha rákattintunk a "Connectivity" menüpont alatt az "USB_OTG_FS"-re, akkor láthatjuk, hogy "Device_Only" módra van állítva. Mivel a számítógéphez szeretnénk csatlakoztatni a fejlesztőpanelt, ezért ebben a rendszerben a PC lesz a host, és az eszközünk a device. Láthatjuk, hogy az egyéb opciók között szerepel a "Host_Only" és "OTG/Dual_Role_Device" is, mindkettő a nevének megfelelő szerepkört képes ellátni.



13. ábra: USB konfiguráció

A fenti képen látható két kipipálható jelölőnégyzet. Az “Activate_VBUS” akkor hasznos, ha meg szeretnénk figyelni, hogy mikor érzékelünk feszültséget eszközünk USB csatlakozóján keresztül. Az “Activate_SOF”, ahol az SOF a Start Of Frame kifejezés rövidítése, minden keret elején impulzust ad a csatlakoztatott eszköznek. A SOF kivezetést általában audioeszközök szinkronizálásához szoktuk felhasználni. Jelen esetben egyiket sem szükséges bekapcsolni, ráadásul a SOF kivezetés nincs is bekötve a fejlesztőpanelen.

A “Parameter Settings” fülön láthatóak az USB perifériához tartozó beállítások, amelyek közül a sebesség (speed) opciót érdemes leginkább észben tartani. Itt tudjuk állítani, hogy az eszközünk milyen elméleti maximális sebességgel végezze az adatcserét.

Következőnek kattintsunk a “Middleware” menüpont alatt az “USB_DEVICE”-ra! Ez a menüpont olyan modulokat, könyvtárakat és beállításokat képes számunkra legenerálni, melyek komplex funkcionalitást valósítanak meg. Később a 22. tananyagrészen részletesebben lesz szó ezekről. Az “USB_DEVICE Mode and Configuration” ablakban láthatjuk, hogy a beállított USB perifériát “Human Interface Device”-ként szeretnénk használni. A “Configuration” ablakban alul, a “Device Descriptor” fül alatt adhatjuk meg az eszköz azonosító adatait. A “PID” (Product ID) és “VID” (Vendor ID)

beállításokat csak nagyon indokolt esetben érdemes módosítani. A Manufacturer és Product stringeket tetszés szerint lehet változtatni, ezt meg is tettük.

Térjünk át a programra! A fenti beállításokkal eredetileg egy, az ST által készített példakódot generál nekünk a fejlesztőkörnyezet. Ennek a példakódnak egy olyan egér az eredménye, amit felismer a számítógép, azonban nem csinál semmit. Ha működésre szeretnénk bírni, akkor azt a "main.c" fájlban tehetjük meg, amit mi megírtunk nektek.

Ha megnyitjuk, akkor azt láthatjuk, hogy készítünk egy struktúrát, inicializáljuk a perifériákat (többek között az USB-t is), majd beolvassuk a gombok állapotát. A struktúra négy byte adatot tartalmaz, ezt fogjuk kiküldeni minden ciklusban a lentebb látható kódsor segítségével.

```
// A mouseHID struktúra adatainak kiküldése  
USBD_HID_SendReport(&hUsbDeviceFS, &mouseHID, sizeof(struct mouse_report_t));
```

A gombok állapotainak beolvasása után lévő értékadások módosításával lehet beállítani, hogy mennyit mozogjon az egér, illetve, hogy melyik gombbal kattintson.

ÖSSZEFOGLALÁS

Ebben a tananyagrészen megismerkedhettek három aszinkron soros kommunikációs protokollal. De mire is jók ezek? Minden bemutatott darab másra. Az UART sebességét tekintve nagyon széles tartományban használjuk és minimális többletinformációt kell alkalmazni az átvitelhez. Kommunikáció esetén overhead-nek hívjuk az adatátvitel azon részét, amely nem a hasznos információt/adatot hordozza, hanem a kommunikációs protokoll működéséhez kell. Az UART mintakódban is használt paraméterekkel a 10 bites keretből 2 bit (start- és stopbit) az, ami nem hasznos adat. Ezzel szemben a CAN esetében a keret jelentős része a vezérlést, és egyéb beépített feladatokat hivatott ellátni. Ez sokkal nagyobb overhead-et jelent, de cserébe kisebb a hibás adat átjutásának esélye, valamint sebességét tekintve (szabványban rögzített maximum: 1Mbit/s) is összemérhető a gyorsabb UART-okkal. Nem elhanyagolható az sem, hogy a CAN-buszon akár 100+ eszköz is kommunikálhat, amíg az UART pont-pont topológiával rendelkezik. A kakukktójs ez esetben az USB, mivel nem egyértelműen összehasonlítható más aszinkron soros protokollokkal sem. Az USB tökéletesen és egyeduralkodó módon kielégíti azt az igényt, amire létrehozták: "Egy csatlakozó mind felett!" Ha univerzálisan felhasználható, gyors kommunikációt szeretnénk létrehozni, nincs is jobb választás nála, azonban mindig figyelembe kell venni, hogy sokkal memóriaigényesebb implementálni, mint a többi társát.

Nemcsak az adat küldésének sebességében és komplexitásában, hanem a résztvevő eszközök számában és távolságában is óriási különbség van. Az UART esetében az elérhető távolság nagyon függ a pontos feszültségszinttől és implementációtól, de az általunk gyakran használt UART-TTL nagyon rövid hatótávval rendelkezik, mely általában kevesebb, mint 2 méter. Ezt sokszorosan túllépi az RS szabványok 12-1200 méteres hatótávval. A CAN busz legfeljebb 1 km hosszú lehet, azonban itt már jelentősen le kell csökkenteni a sebességet, 10 kbit/s-ra. A szabványban rögzített maximum sebességet 40 méterig lehet biztosítani. Az USB pedig 5 méterig biztosít maximális sebességet a 2.0 szabványában

Minden (felsorolt) kommunikációnak megvan a maga előnye és hátránya, amelyekhez az alábbi táblázat segítséget nyújt. Remélem, hogy miután elolvastátok a tananyagot, már Ti is magabiztosan tudtok válogatni és akár változtatni közöttük.

	UART	CAN	USB
Eszközök száma	2	100+	1 master, 127 slave
Felépítés	pont - pont	busz	busz, csillag topológia
Jelek száma	3+2	3	4 - 24
Szabványos sebességtartomány	75 bit/s - 5 Mbit/s	10 kbit/s - 1 Mbit/s	1,5 Mbit/s - 40 Gbit/s
Overhead	20%	40,7%	16,7%
Maximális távolság alacsony sebességgel	1200 m	1000 m	30 m
Maximális távolság legnagyobb sebességgel	~ 2 m	40 m	5 m

2. táblázat - UART, CAN, USB kommunikációk összehasonlítása